APRENDIZADO DE MÁQUINA

Classificação de tumores mamários utilizando Árvore de Decisão

Disciplina: Fundamentos de Inteligência Artificial

Curso: Análise e Desenvolvimento de Sistemas — ULBRA Torres

Integrante: Rian Cardoso Leffa

Data: 17-10-2026

Relatório da Avaliação Parcial 2 (AP2)

1. Introdução e definição do problema

O projeto investiga um problema de classificação: estimar, a partir de características calculadas de uma amostra de tecido mamário, se o registro corresponde a um tumor maligno ou benigno. A distinção é relevante porque erros de classificação em contextos de saúde podem ter consequências importantes. Este trabalho tem finalidade acadêmica e demonstra técnicas de Aprendizado de Máquina; o modelo não deve ser utilizado para diagnóstico clínico ou para orientar decisões médicas.

O objetivo é implementar uma Árvore de Decisão em Python com Scikit-learn e avaliar seu desempenho com um conjunto de teste separado e validação cruzada. O mesmo dataset é utilizado nas etapas de definição, treinamento e avaliação, mantendo a coerência exigida pelo enunciado.

2. Tipo de aprendizado e dataset

Tipo de aprendizado: supervisionado, pois cada registro possui atributos de entrada e um rótulo conhecido (maligno ou benigno). O algoritmo aprende uma relação entre os atributos e os rótulos fornecidos.

Origem: conjunto Breast Cancer Wisconsin Diagnostic, disponibilizado diretamente pela função `load_breast_cancer()` do Scikit-learn. Isso permite reproduzir a execução sem baixar um arquivo externo.

Dimensão: 569 registros e 30 atributos numéricos. As duas classes são maligno e benigno. O conjunto completo contém 212 registros malignos e 357 benignos.

Principais atributos: medidas numéricas calculadas a partir de imagens digitalizadas de amostras, incluindo raio, textura, perímetro, área, suavidade, compacidade, concavidade e simetria, entre outras características. O dataset é educacional e não representa necessariamente todos os contextos clínicos reais.

3. Fluxo do projeto de Machine Learning

Definição: classificar os registros em malignos ou benignos. **Coleta:** carregar o dataset integrado ao Scikit-learn. **Preparação:** separar 80% dos dados para treino e 20% para teste, utilizando estratificação para preservar aproximadamente a proporção das classes. **Treinamento:** ajustar uma Árvore de Decisão com profundidade máxima 4. **Avaliação:** calcular acurácia, precisão, recall, F1-score, matriz de confusão e validação cruzada de cinco partições. **Implantação:** em cenário real, seriam necessários validação clínica independente, avaliação de vieses, proteção de dados, monitoramento e aprovação especializada; a implantação não faz parte deste exercício.

4. Algoritmo clássico: implementação e resultados

Foi utilizada uma Árvore de Decisão (DecisionTreeClassifier), algoritmo supervisionado que aprende regras de divisão dos dados para separar as classes. A profundidade máxima foi limitada a 4 para restringir a complexidade do modelo. A divisão treino/teste utiliza random_state=42 para facilitar a reprodução.

Resultados no conjunto de teste (20% dos dados): acurácia = 0.9386; precisão para a classe maligna = 0.9070; recall/sensibilidade para a classe maligna = 0.9286; F1-score para a classe maligna = 0.9176.

A acurácia representa a proporção total de previsões corretas. A precisão para maligno indica, entre os registros previstos como malignos, a proporção que realmente é maligna. O recall para maligno mede a proporção dos casos malignos reais que o modelo conseguiu identificar. O F1-score combina precisão e recall por meio da média harmônica.

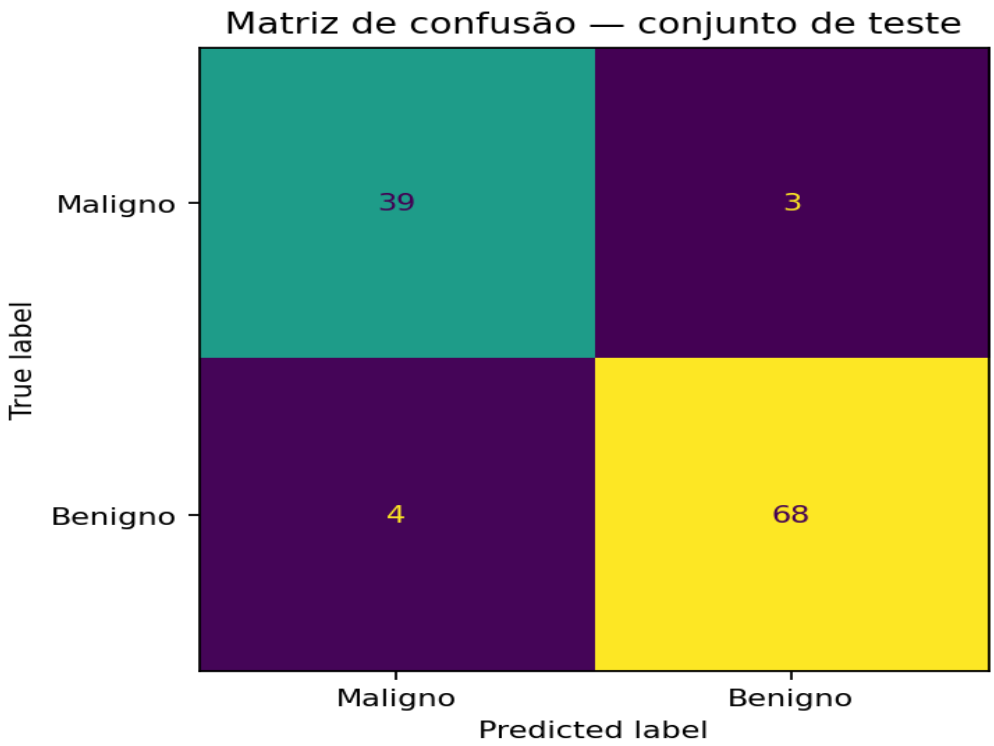


Figura 1 — Matriz de confusão do conjunto de teste. As linhas representam a classe real e as colunas, a classe prevista.

5. Avaliação aprofundada do modelo

Além da avaliação em um conjunto de teste separado, foi aplicada validação cruzada estratificada com cinco partições (5-fold). Em cada rodada, quatro partições são usadas para treinamento e uma para validação; o processo se repete até que cada partição tenha sido utilizada para validar. A estratificação ajuda a manter a proporção das classes em cada partição.

Acurácias por partição: 0.9561, 0.8684, 0.9211, 0.9298, 0.9381. **Média:** 0.9227. **Desvio-padrão:** 0.0295. A média resume o desempenho nas partições, enquanto o desvio-padrão indica quanto os resultados variaram entre elas.

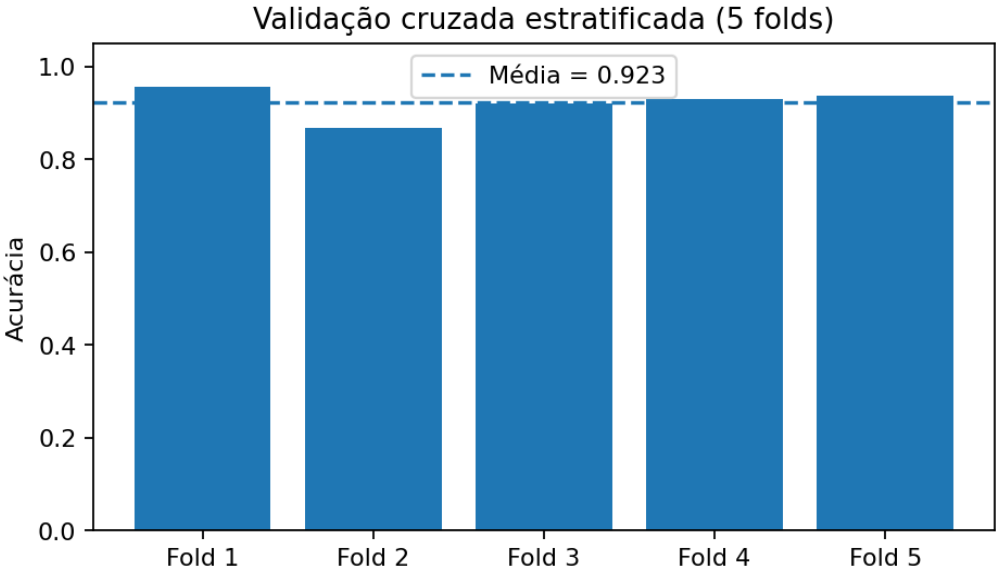


Figura 2 — Acurácia em cada partição da validação cruzada. A linha tracejada representa a média.

A matriz de confusão permite observar os tipos de erro, em vez de apresentar apenas um número geral. Precisão, recall e F1-score são importantes porque diferentes erros podem ter impactos diferentes. Neste

problema, deixar de identificar um caso maligno (falso negativo) merece atenção especial; por isso, o recall da classe maligna deve ser analisado junto com as demais métricas.

6. Análise crítica do desempenho

No teste separado, o modelo alcançou acurácia de 93.9%. Esse resultado, isoladamente, não é suficiente para concluir que o modelo é adequado. A matriz de confusão mostra quantos casos de cada classe foram classificados corretamente e quantos foram confundidos. O recall da classe maligna (92.9%) é especialmente relevante para discutir os casos malignos que o modelo deixou de identificar.

A validação cruzada produziu média de acurácia de 92.3%, com desvio-padrão de 0.030. Comparar essa média com o resultado do teste ajuda a verificar se o desempenho parece consistente em diferentes divisões dos dados. Ainda assim, resultados de validação não garantem generalização para novos hospitais, equipamentos ou populações.

O dataset contém mais registros benignos do que malignos, portanto as classes não estão perfeitamente equilibradas. Em situações desbalanceadas, um modelo pode obter acurácia aparentemente alta favorecendo a classe majoritária e ainda falhar em detectar parte da classe minoritária. Por isso, é necessário observar a matriz de confusão, precisão, recall e F1-score, e não apenas a acurácia.

Como limitações, a Árvore de Decisão pode ser sensível às amostras de treinamento, e a profundidade escolhida é uma configuração inicial, não necessariamente a melhor possível. Uma investigação futura poderia comparar outros algoritmos clássicos, ajustar hiperparâmetros usando validação apropriada e testar dados externos. Qualquer aplicação médica exigiria revisão ética, clínica e regulatória.

7. Conclusão

O projeto aplicou o fluxo de Aprendizado de Máquina supervisionado usando um dataset de classificação binária e uma Árvore de Decisão. A avaliação combinou métricas no conjunto de teste com validação cruzada de cinco partições, fornecendo uma visão mais ampla do desempenho. A principal conclusão é que nenhuma métrica isolada descreve todos os aspectos de um classificador: a acurácia deve ser interpretada junto com a matriz de confusão, precisão, recall e F1-score. A avaliação rigorosa é indispensável antes de considerar qualquer modelo para uso real, especialmente em um domínio sensível como a saúde.

8. Anexo — código-fonte e execução

O código-fonte completo está no arquivo separado projeto_ap2.py, que acompanha este relatório. Ele carrega o dataset, divide os dados, treina o modelo, imprime as métricas e gera os gráficos. Para executar em um ambiente Python, instale Scikit-learn e Matplotlib e rode o arquivo. Também é possível enviar o arquivo ao Google Colab e executá-lo em uma célula.

Dependências: scikit-learn, matplotlib e numpy. **Observação:** o código está no arquivo .py separado para envio ao Google Colab ou GitHub, gerando o link acessível solicitado pelo enunciado.

Anexo A — Código-fonte completo

```
"""
Projeto AP2 – Fundamentos de Inteligência Artificial
Tema: Classificação de tumores mamários com Árvore de Decisão
Dataset: Breast Cancer Wisconsin Diagnostic (incluído no scikit-learn)

Instalação:
    pip install scikit-learn matplotlib numpy

Execução:
    python projeto_ap2.py
"""
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay, classification_report
)

# 1. Carregar os dados
dataset = load_breast_cancer()
X, y = dataset.data, dataset.target
# No dataset: 0 = maligno; 1 = benigno
print("Quantidade de registros:", X.shape[0])
print("Quantidade de atributos:", X.shape[1])
print("Classes:", dataset.target_names, "(atenção: target 0=maligno, 1=benigno)")

# 2. Separar treino e teste, preservando proporção das classes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# 3. Treinar o algoritmo clássico
modelo = DecisionTreeClassifier(random_state=42, max_depth=4)
modelo.fit(X_train, y_train)
previsoes = modelo.predict(X_test)

# 4. Métricas no conjunto de teste
print("\\n--- Avaliação no conjunto de teste ---")
print(f"Acurácia: {accuracy_score(y_test, previsoes):.4f}")
print(f"Precisão para classe maligna: {precision_score(y_test, previsoes, pos_label=0):.4f}")
print(f"Recall/Sensibilidade para classe maligna: {recall_score(y_test, previsoes, pos_label=0):.4f}")
print(f"F1-score para classe maligna: {f1_score(y_test, previsoes, pos_label=0):.4f}")
print("\\nRelatório por classe:")
print(classification_report(
    y_test, previsoes, labels=[0, 1],
    target_names=["maligno", "benigno"], zero_division=0
))

# 5. Matriz de confusão
matriz = confusion_matrix(y_test, previsoes, labels=[0, 1])

print("Matriz de confusão (linhas = real; colunas = previsto; ordem maligno, benigno):")
print(matriz)
ConfusionMatrixDisplay(
    confusion_matrix=matriz, display_labels=["Maligno", "Benigno"]
).plot(values_format="d", colorbar=False)
plt.title("Matriz de confusão – conjunto de teste")
plt.tight_layout()
plt.savefig("matriz_confusao_ap2.png", dpi=160)
plt.show()

# 6. Validação cruzada estratificada com cinco partições
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(
    DecisionTreeClassifier(random_state=42, max_depth=4),
    X, y, cv=cv, scoring="accuracy"
)
print("\\nAcurácia por fold:", [round(s, 4) for s in scores])
print(f"Média da validação cruzada: {scores.mean():.4f}")
print(f"Desvio-padrão: {scores.std():.4f}")

plt.bar([f"Fold {i+1}" for i in range(len(scores))], scores)
plt.axhline(scores.mean(), linestyle="--", label=f"Média = {scores.mean():.3f}")
plt.ylim(0, 1.05)
plt.ylabel("Acurácia")
plt.title("Validação cruzada estratificada (5 folds)")
plt.legend()
plt.tight_layout()
plt.savefig("validacao_cruzada_ap2.png", dpi=160)
plt.show()
```